

Combining Packing and Batching: Faster Amortized Bootstrapping with Polynomial Errors

No Author Given

No Institute Given

Abstract. This work introduces a new bootstrapping algorithm which bootstrapping several messages with polynomial errors growing.

1 Preliminary

1.1 Notations

Denote the set of integers by $\mathbb{Z}$, the set of rational numbers by $\mathbb{Q}$, the set of real numbers by $\mathbb{R}$, and the set of complex numbers by $\mathbb{C}$. For a positive $k \in \mathbb{Z}$, let $[k]$ be the set of integers $\{0, \dots, k-1\}$. We denote $[a, b]$ as the set $\{a, a+1, \dots, b\} \subset \mathbb{Z}$ for any integers $a \leq b$. For a set S and a positive integer n , we denote S^n as the set of all n dimensional vectors with each component belonging to S . A vector is always a column vector by default and is denoted by a bold lower-case letter, e.g., $\mathbf{a}$. We use $\mathbf{a}[i]$ to denote the i -th element of $\mathbf{a}$. For a vector $\mathbf{a} \in \mathbb{Z}^n$, let $\|\mathbf{a}\|_2$, $\|\mathbf{a}\|_\infty$ be its ℓ_2 , ℓ_∞ norm, respectively.

Cyclotomic Ring and Quotient Ring In this work, we only consider the cyclotomic rings with dimension N being two's power, which is widely used in lattice-based cryptography. In this case, the ring is isomorphic to the polynomial ring $\mathcal{R} = \mathbb{Z}[x]/(x^N + 1)$, where each element $a \in \mathcal{R}$ can be expressed as a polynomial in the coefficient representation as $a = a_0 + a_1x + a_2x^2 \cdots + a_{N-1}x^{N-1}$, or $(a_0, \dots, a_{N-1})$ in the vector form. We use $\|\mathbf{a}\|_2$, $\|\mathbf{a}\|_\infty$ to denote the vector's ℓ_2 , ℓ_∞ norms.

For a positive integer Q , let $\mathcal{R}_Q = \mathcal{R}/(Q \cdot \mathcal{R})$ and $\mathbb{Z}_Q = \mathbb{Z}/(Q \cdot \mathbb{Z})$. We use the balanced integer set $[-Q/2, Q/2]$ as a complete set of representatives for coefficients of $\mathcal{R}_Q$ and the quotient ring $\mathbb{Z}_Q$.

Sampling Notations In this work, we use the following notion for the sampling random process: $x \leftarrow \chi$ denotes sampling a random element x from the distribution χ , and $x \leftarrow S$ denotes sampling uniformly from the set S . For the ring setting, $a \leftarrow \chi(\mathcal{R})$ means that every coefficient of $a \in \mathcal{R}$ is sampled from distribution χ . If the context is clear, we may simply write $a \leftarrow \chi$.

Gaussian Distribution For any real number $s > 0$ and an n -dimensional vector $\mathbf{c}$, let $\rho_{s,\mathbf{c}}(\mathbf{x}) := \exp(-\pi\|\mathbf{x} - \mathbf{c}\|^2/s^2)$ be the Gaussian density function with parameter s and centered at $\mathbf{c}$. The discrete gaussian distribution over a lattice coset $\Lambda + \mathbf{u}$ is defined as $D_{\Lambda+\mathbf{u},s,\mathbf{c}}(\mathbf{x}) = \frac{\rho_{s,\mathbf{c}}(\mathbf{x})}{\rho_{s,\mathbf{c}}(\Lambda+\mathbf{u})}$. When $\mathbf{c}$ and $\mathbf{u}$ are both set to be the zero vector and the lattice is clear from the context, we write D_s for short and s is the deviation. In addition, the Gaussian distribution satisfies the Pythagorean additivity for the deviation.

Subgaussian For $\delta > 0$, we call a random variable X over $\mathbb{R}$ is δ -subgaussian with parameter $s > 0$, if for all $t \in \mathbb{R}$, the (scaled) moment-generating function satisfies: $\mathbb{E}[e^{2\pi t X}] \leq e^{\delta \cdot e^{\pi s^2 t^2}}$. If X is a δ -subgaussian with parameter s , then cX is δ -subgaussian with parameter cs for any positive $c \in \mathbb{R}$. Any B -bounded symmetric random variable X (i.e., $|X| \leq B$ always) is 0-subgaussian with parameter $B\sqrt{2\pi}$. Specifically, the uniform distribution over $[-b, b]$ is 0-subgaussian with parameter $b\sqrt{2\pi}$. The gaussian with deviation s is subgaussian with parameter $s\sqrt{2\pi}$. Subgaussians satisfy *Pythagorean additivity* as follow.

Lemma 1.1 ([9]) *For $\delta_i, s_i \geq 0$, and random variables X_i for $i \in [k]$, if X_i is δ_i -subgaussian with parameter s_i conditioning on any values of $X_1, \dots, X_{i-1}$, then $\sum_{i \in [k]} X_i$ is $(\sum_{i \in [k]} \delta_i)$ -subgaussian with parameter $(\sum_{i \in [k]} s_i^2)^{1/2}$.*

The notion of subgaussianity can be extended to vectors: a random real vector $\mathbf{x}$ is δ -subgaussian with parameter r if for all fixed real unit vectors $\mathbf{u}$, the marginal $\langle \mathbf{u}, \mathbf{x} \rangle \in \mathbb{R}$ is δ -subgaussian with parameter r . It is clear from the definition that the concatenation of δ -subgaussian variables or vectors, each of which has a parameter s and is independent of the prior one, is also δ -subgaussian with parameter s . Homogeneity and Pythagorean additivity also hold from linearity of vectors. It is known that the Euclidean norm of the subgaussian random vector has the following upper bound. In this paper, we only need δ to be 0, and thus we write 0-subgaussian simply as subgaussian.

Lemma 1.2 ([6]) *For any subgaussian random variable X with parameter s , we have*

$$\Pr[|X| > t] < 2 \cdot \exp(-\pi t^2/s^2).$$

1.2 MLWE

Our new cryptosystem is based on the module learning with errors, namely MLWE assumption as studied extensively [11]. To describe the problem, we first define the distribution $A_{\mathbf{s},\chi,q}$ over $\mathcal{R}_q^k \times \mathcal{R}_q$. The distribution is parameterized by a secret vector $\mathbf{s} \in \mathcal{R}_q^k$, an error distribution χ over $\mathcal{R}$, and a modulus q . A sample from the distribution $A_{\mathbf{s},\chi}$ is of the form $(b, \mathbf{a}) \in \mathcal{R}_q^{k+1}$ with $b = -\langle \mathbf{s}, \mathbf{a} \rangle + e \bmod q$ where $\mathbf{a} \leftarrow \mathcal{R}_q^k$ and $e \leftarrow \chi$.

Definition 1.3 (MLWE) *For a security parameter λ , let $N := N(\lambda)$ be the degree of the ring, $k = k(\lambda) \geq 1$ be the rank, $q = q(\lambda) \geq 2$ be an integer modulus, and $\chi = \chi(\lambda)$ be an error distribution over $\mathcal{R}$. Given an arbitrary number*

of independent samples from $A_{\mathbf{s}, \chi, q}$, the decision version of MLWE, denoted by $M\text{-DLWE}_{N, k, q, \chi}$, is to distinguish whether the samples come from $A_{\mathbf{s}, \chi, q}$ or the uniform distribution over $\mathcal{R}_q^{k+1}$.

The MLWE problem captures the following two special cases: traditional LWE ($N = 1$) [12] and RLWE ($k = 1$) [8].

The concrete hardness of MLWE can be estimated using the estimator of [2]. We refer the readers to [1] for further details.

1.3 The Batching Algorithm

In this section, we introduce a novel configuration of the underlying cryptosystems for FHEW-like bootstrapping [10], but with notable performance improvements compared to existing state-of-the-art libraries, such as OpenFHE [3, 10] and `tfhe` [4, 5]. We provide extensive experimental results in Sections ?? and ?? to demonstrate the concrete performance enhancements.

Our proposed variant can be viewed as a squared-gadget matrix version of the GSW scheme presented in TFHE [4], specifically by setting $\ell = 1$ as described in their Section 3.2. However, previous works did not recognize the potential advantages of this configuration. Consequently, they considered different values for ℓ and optimized accordingly, such as TFHE with $\ell = 3$ and FHEW with $\ell = 3$. This work identifies how to set FHE parameters along with a faster implementation under this specific configuration.

Rather than using the torus representation, we choose to describe the scheme in the integer setting, which offers easier readability and understanding. As we later aim to optimize the FHE parameters, we provide a detailed analysis of the noise growth and computational complexity of the homomorphic operations within the scheme. Interestingly, this configuration of the scheme can be viewed as a scale-based GSW/LWE system utilizing two moduli. This bears some conceptual similarities to the second generation BFV [7], while benefiting from the noise growth advantage of the third generation (e.g., GSW).

Now we present our variant based on the module learning with errors, with two basic encryption schemes called `Mat-MGSW` and `Vec-MLWE`. Our new insight is to identify that our scale-based external product can be implemented more efficiently than the prior configurations. For this work, the symmetric-key version of the cryptosystem suffices to present our insights and applications, and we notice that the public-key version just works analogously.

Parameters Let k be the module rank of MLWE and $T > Q$ be two moduli. Our basic scheme includes a variant of MGSW system under modulus T and a variant of MLWE system. Let $\mathcal{R} = \mathbb{Z}[x]/(x^N + 1)$ be a cyclotomic ring, where N is a two's power.

Construction The cryptosystem works as follow.

- `KeyGen`(1^λ): Choose randomly $\mathbf{S} \leftarrow \chi^{k \times r}$ and set $\text{sk} := \begin{pmatrix} \mathbf{I} \\ \mathbf{S} \end{pmatrix} \in \mathcal{R}^{(k+r) \times r}$.

- **Mat-MGSW.Enc**(sk, $\mathbf{M} \in \mathcal{R}^{r \times r}$): Given input sk and message $\mathbf{M}$, the algorithm performs the following steps. Sample a uniformly random matrix $\mathbf{A} \leftarrow \mathcal{R}_T^{k \times (k+r)}$ and a small noise matrix $\mathbf{E} \leftarrow \psi$, where ψ is a distribution over $\mathcal{R}^{r \times (k+r)}$. The ciphertext $\mathbf{C}$ is set as

$$\begin{pmatrix} -\mathbf{S}^\top \mathbf{A} + \mathbf{E} \\ \mathbf{A} \end{pmatrix} + \left\lceil \frac{T}{Q} \cdot \begin{pmatrix} \mathbf{M} & \mathbf{M} \cdot \mathbf{S}^\top \\ \mathbf{0} & \mathbf{0} \end{pmatrix} \right\rceil \pmod T.$$

- **Vec-MLWE.Enc**(sk, $\mathbf{m} \in \mathcal{R}^r$, t): Given input sk, message $\mathbf{m}$, and an additional parameter t , the algorithm does the following. Sample a uniformly random vector $\mathbf{a} \leftarrow \mathcal{R}_Q^k$, and compute

$$\mathbf{c} := \begin{pmatrix} \mathbf{b} \\ \mathbf{a} \end{pmatrix} = \begin{pmatrix} -\mathbf{S}^\top \cdot \mathbf{a} + \mathbf{e} + \mathbf{m} \cdot Q/t \\ \mathbf{a} \end{pmatrix} \pmod Q.$$

Then output ct as $\mathbf{c} \in \mathcal{R}_T^{k+r}$ whose coefficients are in $[-Q/2, Q/2]$.

- **Scale-based External Product**. Given a Mat-MGSW ciphertext $\mathbf{C} \in \mathcal{R}_T^{(k+r) \times (k+r)}$ and a Vec-MLWE ciphertext $\mathbf{c} \in \mathcal{R}_T^{k+r}$ (whose coefficients are in $[-Q/2, Q/2]$) as input, the external product $\boxtimes$ works as follow:

$$\mathbf{C} \boxtimes \mathbf{c} = \left\lfloor \mathbf{C} \cdot \mathbf{c} \cdot \frac{Q}{T} \right\rfloor \pmod Q,$$

where $\mathbf{C} \cdot \mathbf{c}$ is computed under modulo T , and the division and rounding are applied coefficient-wise. That is, $\lfloor Q \cdot a/T \rfloor$ is computed by first interpreting a in the coefficient embedding, i.e., $\sum a_i x^i$, and then outputting $a' = \sum \lfloor Q \cdot a_i/T \rfloor x^i$.

Note: The result of the external product is still stored as a vector in $\mathcal{R}_T^{k+r}$, but note that the coefficients are in $[-Q/2, Q/2]$.

- **Vec-MLWE.Dec**(sk, $\mathbf{c} \in \mathcal{R}_T^{k+r}$, t): Given input sk, ciphertext $\mathbf{c}$, and an additional parameter t , output $\left\lfloor \frac{t}{Q} \cdot (\mathbf{sk}^\top \cdot \mathbf{c} \pmod Q) \right\rfloor \pmod t$.

Remark 1.4 *The Mat-MGSW/Vec-MLWE ciphertext is stored by matrices/vectors over $\mathcal{R}_T$, respectively. Each ring element of Vec-MLWE ciphertexts has coefficients represented in $[-Q/2, Q/2]$.*

The CPA-security of the encryption scheme follows directly from the hardness of MLWE as analyzed in many prior works [11]. The correctness of the Mat-MGSW scheme is with respect to modulo Q , and that of Vec-MLWE is with respect to modulo t specified in the input to the decryption algorithm. Using this additional parameter t allows the integration with the FHEW [6] framework, where NAND can be computed by using only homomorphic addition, but the underlying modulus would be changed. This idea has been used before [6], so we refer the readers the prior work for further intuitions. The plaintext computation of

external product takes each entry of $\mathbf{M}_1, \mathbf{m}_2 \in \mathcal{R}$ and outputs $\mathbf{M}_1 \cdot \mathbf{m}_2 \bmod t$. As setting $t|Q$ suffices for our next applications of bootstrapping (Section ??), we just consider this case in this section for ease of the analysis. Next we formally analyze the correctness and noise growth of the scale-based external product.

We notice that even though the plaintext moduli of the Mat-MGSW and Vec-MLWE are different, their conjunction suffices to improve the FHEW-like bootstrapping as the message space required is only the monomials, i.e., $\{1, x, x^2, \dots, x^{2^N-1}\}$. This is clearly supported by both the Mat-MGSW and Vec-MLWE systems. We present more details in Section ??.

To analyze correctness and error growth, we first re-define the following notions for the error terms of Mat-MGSW/Vec-MLWE ciphertexts under our scale-based scheme.

Definition 1.5 *We use the symbols $\text{Mat-MGSW}(\mathbf{M}_1)$ and $\text{Vec-MLWE}^t(\mathbf{m}_2)$ to present the set of all valid Mat-MGSW ciphertexts encrypting $\mathbf{M}_1$ and the set of all valid Vec-MLWE ciphertexts encrypting $\mathbf{m}_2 \bmod t$, respectively. We put a subscript $\mathbf{s}$, i.e., $\text{Vec-MLWE}_{\mathbf{s}}^t(\mathbf{m})$ to indicate the secret key associated with the ciphertext. If the context is clear, we may omit the subscript for simplicity.*

Note that for Vec-MLWE encryption and decryption, there is an additional parameter t specified in the input.

Definition 1.6 *Let $\mathbf{C}$ be a Mat-MGSW ciphertext encrypting $\mathbf{M}_1$ and $\mathbf{c}$ be a Vec-MLWE ciphertext encrypting $\mathbf{m}_2$. Define the error function $\text{Err}(\cdot, \cdot)$ that works as follow: on input a Mat-MGSW ciphertext $\mathbf{C} \in \mathcal{R}_T^{(k+r) \times (k+r)}$ and the message $\mathbf{M}_1$ (or a Vec-MLWE ciphertext $\mathbf{c} \in \mathcal{R}_T^{(k+r)}$ and the message $\mathbf{m}_2$) outputs the error term $\mathbf{E}_1 = \text{sk} \cdot (\mathbf{C} - \mathbf{M}_1 \cdot T/Q) \bmod T$ (or respectively, $\mathbf{e}_2 = \text{sk} \cdot (\mathbf{c} - (Q/t \cdot \mathbf{m}_2, 0, \dots, 0)^\top) \bmod Q$).*

Note that we use the input type to differentiate whether $\text{Err}(\cdot, \cdot)$ should output a Mat-MGSW error term or a Vec-MLWE error term.

Now, we analyze the correctness and the noise growth of the external product in the following lemma. We adopt the notations from the construction.

Lemma 1.7 *Let $\mathbf{C}_1 \in \text{Mat-MGSW}(\mathbf{M}_1)$ with error term $\mathbf{E}_1 = \text{Err}(\mathbf{C}_1, \mathbf{M}_1)$, $\mathbf{c}_2 \in \text{Vec-MLWE}^t(\mathbf{m}_2)$ with error term $\mathbf{e}_2 = \text{Err}(\mathbf{c}_2, \mathbf{m}_2)$, and $\mathbf{c}_\square = \mathbf{C}_1 \square \mathbf{c}_2$ be the external product (all under the same secret key). Then we have:*

- **Correctness.** $\mathbf{c}_\square \in \text{Vec-MLWE}^t(\mathbf{M}_1 \cdot \mathbf{m}_2 \bmod t)$ (i.e., by treating each entry of $\mathbf{M}_1, \mathbf{m}_2 \in \mathcal{R}$ and then outputting $\mathbf{M}_1 \cdot \mathbf{m}_2 \bmod t$), with an error term $\mathbf{e}_\square$ whose norm can be upper bounded as follows.
- **Noise growth.** For the noise term $\mathbf{e}_\square$, we have:
 - Worst case: The infinity norm can be bounded as below.

$$\|\mathbf{e}_\square\|_\infty \leq B_1 + B_2 + B_{\text{sk}}$$

where the auxiliary parameters are defined as follows:

$$* B_1 := \frac{(k+r)NQ^2\|\mathbf{E}_1\|_\infty}{2T}$$

- * $B_2 := \|\mathbf{M}_1 \cdot \mathbf{e}_2\|_\infty$
- * $B_{\text{sk}} := \frac{kNQ^2 + Q^2N}{4T} + \frac{kN\|\text{sk}\|_\infty + 1}{2}$
- *Average case: Suppose each coefficient of $\mathbf{E}_1$ is subgaussian distribution with parameter σ_1 , each coefficient of $\mathbf{e}_2$ is subgaussian distribution with parameter σ_2 , and $\mathbf{E}_1$ is independent of $\mathbf{e}_2$. Then, we have that $\mathbf{e}_\square$ is a subgaussian distribution with parameter less than:*

$$\sqrt{\frac{Q^2}{T^2} \cdot \gamma_1^2 + \gamma_2^2 + \frac{Q^2}{T^2} \cdot \gamma_3^2 + \gamma_4^2}$$

where the auxiliary parameters are defined as follows:

- * $\gamma_1 := Q\sigma_1\sqrt{(k+r)N}/2$;
- * $\gamma_2 := \|\mathbf{M}_1\|_\infty\sigma_2\sqrt{kN}$;
- * $\gamma_3 := \sqrt{2\pi kNQ^2 + 2\pi NQ^2}/4$;
- * $\gamma_4 := \sqrt{2\pi kN\|\text{sk}\|_\infty^2 + 2\pi}/2$.

Application-based Optimizations In our application of the FHEW-like bootstrapping, we can use the following facts to simplify and optimize the bound. (1) The parameters are set such that $\Delta \cdot Q^2 < T$ for some integer Δ . (2) In the bootstrapping process, the message $\mathbf{M}_1$ is always a diagonal matrix with monomial non-zero entries, and thus $\|\mathbf{M}_1\|_\infty = 1$. (3) We use the uniform ternary secret. (4) The distribution of $\mathbf{C}_1$ is uniformly random (computationally), and that of $\mathbf{c}_2$ can be assumed uniformly random, empirically [6].

Adding these conditions in the above lemma results in the following corollary.

Corollary 1.8 *Adopt the notions from Lemma 1.7. Suppose that each coefficient of $\mathbf{C}_1$ is uniform over $[-T/2, T/2]$ and each coefficient of $\mathbf{c}_2$ is uniform over $[-Q/2, Q/2]$. If the diagonal matrix $\mathbf{M}_1$ has only monomial non-zero entries and $\Delta \cdot Q^2 \leq T$, then $\mathbf{e}_\square$ is subgaussian with parameter less than:*

$$\sqrt{\frac{(k+r)N\sigma_1^2}{4\Delta^2} + \sigma_2^2 + \frac{3\pi + 2\pi kN + 12\pi\Delta^2 + 8\pi kN\Delta^2}{72\Delta^2}}.$$

1.4 RNS-Operations for homomorphic encryption

ModUp(c)

ModDown(c)

2 Batch Bootstrapping with algebraic base

We start with "Packing Monomial Polynomial MULtiplication":

Algorithm 1 Packing MPMUL-P – MPMUL($\vec{c}, \vec{C}, \text{KSK}$)

Input:

- 1: A list $\vec{c} = (c_0, \dots, c_{n-1})$, where for each $0 \leq i \leq n-1$, $c_i \in \text{RLWE}(\sum_{j=1}^r X^{u_j[i]} \cdot \alpha_j)$;

Input:

- 2: A list $\vec{C} = (C_1, \dots, C_{\log B-1})$, where for each $1 \leq i \leq \log B-1$, $C_i \in \text{RGSW}(v_i)$, $v = \sum_{i=0}^{\log B-1} v_i \cdot 2^i$ and $v \leq B$;

Input:

- 3: some necessary key-switch key KSK .

Output: A list $\vec{c} = (c_1, \dots, c_{n-1})$, where for each $0 \leq i \leq n-1$, $c_i \in \text{RLWE}(\sum_{j=1}^r X^{w_j[i] \cdot \alpha_j})$, $w_j = u_j \cdot Y^v$

- 4: **for** $i = 1$ to $\log B - 1$ **do**
 - 5: **for** $j = 0$ to $2^i - 1$ **do**
 - 6: $\hat{c}_j \leftarrow (\text{EvalAuto}(c_{n-2^i+j}, -1) - c_j) \boxplus C_i + c_j$
 - 7: **end for**
 - 8: **for** $j = 2^i$ to $n-1$ **do**
 - 9: $\hat{c}_j \leftarrow (c_{j-2^i} - c_j) \boxplus C_i + c_j$
 - 10: **end for**
 - $(c_0, \dots, c_{n-1}) = (\hat{c}_0, \dots, \hat{c}_{n-1})$
 - 11: **end for**
 - 12: **return** $(c_0, \dots, c_{n-1})$
-

The following is the packing "BIN-SAB"

Algorithm 2 Packing BIN-SAB

Input:

- 1: r RLWE ciphertexts $(a_1, b_1), \dots, (a_r, b_r)$, for each $1 \leq i \leq r$, (a_i, b_i) encrypting m_i with error e_i under the same secret s

Input:

- 2: RGSW ciphertexts encrypting the binary decomposition of $\mathbf{d} = \text{diff}(s)$: $\mathbf{C}_{i,j} \in \text{RGSW}(d_{i,j})$, for $0 \leq i \leq h$, $0 \leq h \leq \log B - 1$, $\sum_{j=0}^{\log B - 1} d_{i,j} \cdot 2^j = \mathbf{d}[i] \in \text{diff}(s)$

Input:

- 3: Some necessary key-switch key KSK and test vectors $\{t_{ij}\}$

Output: A list $\vec{\mathcal{C}} = (\mathbf{c}_1, \dots, \mathbf{c}_{n-1})$, where for each $0 \leq i \leq n - 1$, $\mathbf{c}_i \in \text{RLWE}(\sum_{j=1}^r t_{ij} \cdot X^{m_j[i] + e_j[i]} \cdot \alpha_j)$.

- 4: $\vec{\mathcal{C}} = (\mathbf{c}_0, \dots, \mathbf{c}_{n-1})$, where $\mathbf{c}_i \in \text{RLWE}(\sum_{j=1}^r t_{ij} \cdot X^{b_j[i]} \cdot \alpha_j)$
 - 5: **for** $i = 1$ **to** $h - 1$ **do**
 - 6: Let $\vec{\mathcal{C}} = (\mathbf{C}_{i,1}, \dots, \mathbf{C}_{i,\log n - 1})$
 - 7: $\vec{\mathcal{C}} = \text{P} - \text{MPMUL}(\vec{\mathcal{C}}, \vec{\mathcal{C}}, \text{KSK})$
 - 8: **for** $k = 0$ **to** $n - 1$ **do**
 - 9: $\text{Hom-Tr}(\mathbf{c}_i \cdot \sum_{j=1}^r X^{-a_j[k]} \cdot \alpha_j^\vee \cdot \beta_j)$
 - 10: **end for**
 - 11: $\vec{\mathcal{C}} = (\mathbf{c}_1, \dots, \mathbf{c}_{n-1})$
 - 12: **end for**
 - 13: Let $\vec{\mathcal{C}} = (\mathbf{C}_{h,1}, \dots, \mathbf{C}_{h,\log n - 1})$
 - 14: $\vec{\mathcal{C}} = \text{P} - \text{MPMUL}(\vec{\mathcal{C}}, \vec{\mathcal{C}}, \text{KSK})$
 - 15: **return** $(\mathbf{c}_0, \dots, \mathbf{c}_{n-1})$
-

3 Batch Bootstrapping with Vec-LWE – Mat-MGSW

Parameters We begin with a description of the parameters:

- n : degree of input RLWE, the underlying ring is $\mathbb{Z}_q[Y]/(Y^n + 1)$.
- r : the number of messages of one input RLWE ciphertext.
- q : the modulus of the input RLWE scheme.
- $\mathcal{R}$: $\mathbb{Z}[x]/(x^N + 1)$ for some power-of-two N , the underlying ring for BTS.
- k : the rank of the Vec-MLWE and Mat-MGSW scheme.
- T : the modulus of the scale-based Mat-MGSW scheme.
- Q : the modulus of the Vec-MLWE scheme in the scale-based external product.
We set $\Delta \cdot Q^2 < T$ for some integer Δ .

Algorithm 3 Packing MPMUL-P – MPMUL($\vec{c}, \vec{C}, \text{KSK}$)

Input:

- 1: A list $\vec{c} = (c_0, \dots, c_{n-1})$, where for each $0 \leq i \leq n-1$, $c_i \in \text{Vec-MLWE}(X^{u_1[i]}, \dots, X^{u_r[i]})$;

Input:

- 2: A $\vec{C} = (C_1, \dots, C_{\log B-1})$, where for each $1 \leq i \leq \log B-1$, $C_i \in \text{Mat-MGSW}(v_{0i}, \dots, v_{ri})$, $v_i = \sum_{j=0}^{\log B-1} v_{ij}$ and $\max_{1 \leq i \leq r} v_i \leq B$;

Input:

- 3: some necessary key-switch key KSK .

Output: A list $\vec{c} = (c_1, \dots, c_{n-1})$, where for each $0 \leq i \leq n-1$, $c_i \in \text{RLWE}(\sum_{j=1}^r X^{w_j[i] \cdot \alpha_j})$, $w_j = u_j \cdot Y^{v_j}$.

- 4: **for** $i = 1$ to $\log B - 1$ **do**
 - 5: **for** $j = 0$ to $2^i - 1$ **do**
 - 6: $\hat{c}_j \leftarrow (\text{EvalAuto}(c_{n-2^i+j}, -1) - c_j) \boxplus C_i + c_j$
 - 7: **end for**
 - 8: **for** $j = 2^i$ to $n-1$ **do**
 - 9: $\hat{c}_j \leftarrow (c_{j-2^i} - c_j) \boxplus C_i + c_j$
 - 10: **end for**
 - $(c_0, \dots, c_{n-1}) = (\hat{c}_0, \dots, \hat{c}_{n-1})$
 - 11: **end for**
 - 12: **return** $(c_0, \dots, c_{n-1})$
-

The following is the packing "BIN-SAB"

Algorithm 4 Packing BIN-SAB

Input:

- 1: r RLWE ciphertexts $(a_1, b_1), \dots, (a_r, b_r)$, for each $1 \leq j \leq r$, (a, b_j) encrypting m_j with error e_j under the secret s_j

Input:

- 2: RGSW ciphertexts encrypting the binary decomposition of $\mathbf{d}^{(j)} = \text{diff}(s_j)$: $\mathbf{C}_{i,\ell} \in \text{RGSW}(\text{Diag}(d_{i,\ell}^{(1)}, \dots, d_{i,\ell}^{(r)}))$, for $0 \leq i \leq h$, $1 \leq j \leq r$, $0 \leq \ell \leq \log B - 1$, $\sum_{\ell=0}^{\log B-1} d_{i,\ell}^{(j)} \cdot 2^\ell = \mathbf{d}^{(j)}[i] \in \text{diff}(s_j)$

Input:

- 3: Some necessary key-switch key KSK and test vectors $\{t_{ij}\}$

Output: A list $\vec{\mathcal{C}} = (\mathbf{c}_1, \dots, \mathbf{c}_{n-1})$, where for each $0 \leq i \leq n-1$, $\mathbf{c}_i \in \text{Vec-MLWE}(t_{i1} \cdot X^{m_1[i]+e_1[i]}, \dots, t_{ir} \cdot X^{m_r[i]+e_r[i]})$.

- 4: $\vec{\mathcal{C}} = (\mathbf{c}_0, \dots, \mathbf{c}_{n-1})$, where $\mathbf{c}_i \in \text{Vec-MLWE}(t_{i1} \cdot X^{b_1[i]}, \dots, t_{ir} \cdot X^{b_r[i]})$
 - 5: **for** $i = 1$ to $h-1$ **do**
 - 6: Let $\vec{\mathcal{C}} = (\mathbf{C}_{i,1}, \dots, \mathbf{C}_{i,\log n-1})$
 - 7: $\vec{\mathcal{C}} = \text{P} - \text{MPMUL}(\vec{\mathcal{C}}, \vec{\mathcal{C}}, \text{KSK})$
 - 8: **for** $k = 0$ to $n-1$ **do**
 - 9: $\mathbf{c}_k = \mathbf{c}_k \cdot X^{a_k}$
 - 10: **end for**
 - 11: $\vec{\mathcal{C}} = (\mathbf{c}_1, \dots, \mathbf{c}_{n-1})$
 - 12: **end for**
 - 13: Let $\vec{\mathcal{C}} = (\mathbf{C}_{h,1}, \dots, \mathbf{C}_{h,\log n-1})$
 - 14: $\vec{\mathcal{C}} = \text{P} - \text{MPMUL}(\vec{\mathcal{C}}, \vec{\mathcal{C}}, \text{KSK})$
 - 15: **return** $(\mathbf{c}_0, \dots, \mathbf{c}_{n-1})$
-

References

1. M. Albrecht, M. Chase, H. Chen, J. Ding, S. Goldwasser, S. Gorbunov, S. Halevi, J. Hoffstein, K. Laine, K. Lauter, et al. Homomorphic encryption standard. In *Protecting Privacy through Homomorphic Encryption*, pages 31–62. Springer, 2021.
2. M. R. Albrecht, R. Player, and S. Scott. On the concrete hardness of learning with errors. *J. Mathematical Cryptology*, 9(3):169–203, 2015.
3. A. A. Badawi, J. Bates, F. Bergamaschi, D. B. Cousins, S. Erabelli, N. Genise, S. Halevi, H. Hunt, A. Kim, Y. Lee, Z. Liu, D. Micciancio, I. Quah, Y. Polyakov, S. R.V., K. Rohloff, J. Saylor, D. Suponitsky, M. Triplett, V. Vaikuntanathan, and V. Zucca. Openfhe: Open-source fully homomorphic encryption library. Cryptology ePrint Archive, Paper 2022/915, 2022. <https://eprint.iacr.org/2022/915>.
4. I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène. TFHE: Fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, Jan. 2020.
5. I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène. Tfhf: Fast fully homomorphic encryption library. Github, 2023. <https://github.com/tfhf/tfhf>.
6. L. Ducas and D. Micciancio. FHEW: Bootstrapping homomorphic encryption in less than a second. In E. Oswald and M. Fischlin, editors, *EUROCRYPT 2015, Part I*, volume 9056 of *LNCS*, pages 617–640. Springer, Berlin, Heidelberg, Apr. 2015.

7. J. Fan and F. Vercauteren. Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Paper 2012/144, 2012. <https://eprint.iacr.org/2012/144>.
8. V. Lyubashevsky, C. Peikert, and O. Regev. On ideal lattices and learning with errors over rings. In H. Gilbert, editor, *EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 1–23. Springer, Berlin, Heidelberg, May / June 2010.
9. V. Lyubashevsky, C. Peikert, and O. Regev. A toolkit for ring-LWE cryptography. In T. Johansson and P. Q. Nguyen, editors, *EUROCRYPT 2013*, volume 7881 of *LNCS*, pages 35–54. Springer, Berlin, Heidelberg, May 2013.
10. D. Micciancio and Y. Polyakov. Bootstrapping in fhe-like cryptosystems. In *WAHC '21: Proceedings of the 9th on Workshop on Encrypted Computing & Applied Homomorphic Cryptography, Virtual Event, Korea, 15 November 2021*, pages 17–28. WAHC@ACM, 2021.
11. C. Peikert and Z. Pepin. Algebraically structured LWE, revisited. In D. Hofheinz and A. Rosen, editors, *TCC 2019, Part I*, volume 11891 of *LNCS*, pages 1–23. Springer, Cham, Dec. 2019.
12. O. Regev. On lattices, learning with errors, random linear codes, and cryptography. In H. N. Gabow and R. Fagin, editors, *37th ACM STOC*, pages 84–93. ACM Press, May 2005.